

Punto 4.3 - AsNoTracking y AsNoTrackingWithIdentityResolution

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren el comportamiento de AsNoTracking y AsNoTrackingWithIdentityResolution. Comparar el número de instancias duplicadas, el tiempo de ejecución y el SQL generado. Aplicar la variante adecuada según el escenario.

Contexto del proyecto: En el punto 4.2 se estudió el control del tracking, comparando las consultas con Tracking y sin Tracking. En este punto se profundiza en las variantes de No Tracking, que permiten resolver las instancias duplicadas en consultas con relaciones. Esta técnica se usará en el punto 4.4 para el problema N+1.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de resolución de identidad a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConPlanchasAsNoTracking();
    List<OrdenFabricacion> ObtenerConPlanchasConResolucionIdentidad();
    int ContarPlanchasInstanciadas(List<OrdenFabricacion> ordenes);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConPlanchasAsNoTracking(); → método que carga las órdenes con planchas usando AsNoTracking.

Línea 11: List ObtenerConPlanchasConResolucionIdentidad(); → método que carga las órdenes con planchas usando AsNoTrackingWithIdentityResolution.

Línea 12: int ContarPlanchasInstanciadas(List ordenes); → método que cuenta las planchas instanciadas.

Línea 14: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 15: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```

public List<OrdenFabricacion> ObtenerConPlanchasAsNoTracking()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConPlanchasConResolucionIdentidad()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public int ContarPlanchasInstanciadas(List<OrdenFabricacion> ordenes)
{
    var instancias = new HashSet<PlanchaAcero>(ReferenceEqualityComparer.Instance);
    foreach (var orden in ordenes)
    {
        foreach (var plancha in orden.Planchas)
        {
            instancias.Add(plancha);
        }
    }
    return instancias.Count;
}

```

Línea 1: `public List ObtenerConPlanchasAsNoTracking()` → declara el método.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 6: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 7: `.ToList();` → materializa la consulta.

Línea 10: `public List ObtenerConPlanchasConResolucionIdentidad()` → declara el método.

Línea 12: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 13: `.AsNoTrackingWithIdentityResolution()` → aplica `AsNoTrackingWithIdentityResolution`.

Línea 14: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 15: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 16: `.ToList();` → materializa la consulta.

Línea 19: `public int ContarPlanchasInstanciadas(List ordenes)` → declara el método.

Línea 21: `var instancias = new HashSet(ReferenceEqualityComparer.Instance);` → crea un conjunto de instancias por referencia.

Línea 22: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 24: `foreach (var plancha in orden.Planchas)` → itera sobre las planchas.

Línea 26: `instancias.Add(plancha);` → añade la plancha al conjunto.

Línea 29: `return instancias.Count;` → devuelve el número de instancias únicas.

Error común: si se usa `HashSet` sin `ReferenceEqualityComparer`, el conjunto usa `Equals` y `GetHashCode` de la entidad, que pueden no estar sobrescritos. Se debe usar `ReferenceEqualityComparer.Instance` para comparar por referencia.

Paso 4: Crear el caso de uso de resolución de identidad

Crear el archivo `src/AceriaData.Application/UseCases/ResolucionIdentidadUseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ResolucionIdentidadUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ResolucionIdentidadUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== ASNOTRACKING VS ASNOTRACKINGWITHIDENTITYRESOLUTION ===");

        DemostrarAsNoTracking();
        DemostrarConResolucionIdentidad();
        CompararRendimiento();
    }

    private void DemostrarAsNoTracking()
    {
        Console.WriteLine("\n--- AsNoTracking ---");

        var ordenes = _unidad.Ordenes.ObtenerConPlanchasAsNoTracking();
        var instancias = _unidad.Ordenes.ContarPlanchasInstanciadas(ordenes);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        Console.WriteLine($"Instancias de planchas: {instancias}");
    }

    private void DemostrarConResolucionIdentidad()
    {
        Console.WriteLine("\n--- AsNoTrackingWithIdentityResolution ---");

        var ordenes = _unidad.Ordenes.ObtenerConPlanchasConResolucionIdentidad();
        var instancias = _unidad.Ordenes.ContarPlanchasInstanciadas(ordenes);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        Console.WriteLine($"Instancias de planchas: {instancias}");
    }

    private void CompararRendimiento()
    {
        Console.WriteLine("\n--- Comparación de rendimiento ---");

        var cronometroNoTracking = Stopwatch.StartNew();
        var ordenesNoTracking = _unidad.Ordenes.ObtenerConPlanchasAsNoTracking();
        cronometroNoTracking.Stop();

        var cronometroResolucion = Stopwatch.StartNew();
        var ordenesResolucion = _unidad.Ordenes.ObtenerConPlanchasConResolucionIdentidad();
```

```

        cronometroResolucion.Stop();

        Console.WriteLine($"AsNoTracking: {cronometroNoTracking.ElapsedMilliseconds} ms | Órdenes:
{ordenesNoTracking.Count}");
        Console.WriteLine($"AsNoTrackingWithIdentityResolution:
{cronometroResolucion.ElapsedMilliseconds} ms | Órdenes: {ordenesResolucion.Count}");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ResolucionIdentidadUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public ResolucionIdentidadUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== ASNOTRACKING VS ASNOTRACKINGWITHIDENTITYRESOLUTION
==="); → muestra la cabecera.

Línea 19: DemostrarAsNoTracking(); → llama al método de AsNoTracking.

Línea 20: DemostrarConResolucionIdentidad(); → llama al método de resolución de identidad.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 24: private void DemostrarAsNoTracking() → declara el método.

Línea 26: Console.WriteLine("\n--- AsNoTracking ---"); → muestra la cabecera.

Línea 28: var ordenes = _unidad.Ordenes.ObtenerConPlanchasAsNoTracking(); → carga las órdenes con AsNoTracking.

Línea 29: var instancias = _unidad.Ordenes.ContarPlanchasInstanciadas(ordenes); → cuenta las instancias de planchas.

Línea 31: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 32: Console.WriteLine(\$"Instancias de planchas: {instancias}"); → muestra el número de instancias.

Línea 35: private void DemostrarConResolucionIdentidad() → declara el método.

Línea 37: Console.WriteLine("\n--- AsNoTrackingWithIdentityResolution ---"); → muestra la cabecera.

Línea 39: var ordenes = _unidad.Ordenes.ObtenerConPlanchasConResolucionIdentidad(); → carga las órdenes con resolución de identidad.

Línea 40: var instancias = _unidad.Ordenes.ContarPlanchasInstanciadas(ordenes); → cuenta las instancias de planchas.

Línea 42: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 43: Console.WriteLine(\$"Instancias de planchas: {instancias}"); → muestra el número de instancias.

Línea 46: private void CompararRendimiento() → declara el método.

Línea 48: Console.WriteLine("\n--- Comparación de rendimiento ---"); → muestra la cabecera.

Línea 50: var cronometroNoTracking = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 51: `var ordenesNoTracking = _unidad.Ordenes.ObtenerConPlanchasAsNoTracking();` → carga las órdenes con `AsNoTracking`.

Línea 52: `cronometroNoTracking.Stop();` → detiene el cronómetro.

Línea 54: `var cronometroResolucion = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 55: `var ordenesResolucion = _unidad.Ordenes.ObtenerConPlanchasConResolucionIdentidad();` → carga las órdenes con resolución de identidad.

Línea 56: `cronometroResolucion.Stop();` → detiene el cronómetro.

Línea 58: `Console.WriteLine($"AsNoTracking: {cronometroNoTracking.ElapsedMilliseconds} ms | Órdenes: {ordenesNoTracking.Count}");` → muestra el tiempo de `AsNoTracking`.

Línea 59: `Console.WriteLine($"AsNoTrackingWithIdentityResolution: {cronometroResolucion.ElapsedMilliseconds} ms | Órdenes: {ordenesResolucion.Count}");` → muestra el tiempo de resolución de identidad.

Error común: si se ejecutan las dos consultas en el mismo `DbContext`, la caché de identidad del Change Tracker puede afectar a los resultados. Se debe usar un `DbContext` distinto para cada consulta.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ResolucionIdentidadUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ResolucionIdentidadUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con planchas compartidas

Asegurarse de que hay datos en la base de datos con planchas que aparezcan en varias órdenes:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
    context OrdenesFabricacion.AddRange(orden1, orden2, orden3);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    var plancha4 = new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);
    context.SaveChanges();
}

```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 10: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);` → registra las órdenes.

Línea 11: `context.SaveChanges();` → inserta las órdenes.

Línea 13: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 14: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 15: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 16: `var plancha4 = new PlanchaAcero { ... };` → crea la cuarta plancha.

Línea 17: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3, plancha4);` → registra las planchas.

Línea 18: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

| Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el número de instancias de planchas con `AsNoTracking` y con `AsNoTrackingWithIdentityResolution`.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== ASNOTRACKING VS ASNOTRACKINGWITHIDENTITYRESOLUTION ===

--- AsNoTracking ---
Órdenes: 3
Instancias de planchas: 4

--- AsNoTrackingWithIdentityResolution ---
Órdenes: 3
Instancias de planchas: 4

--- Comparación de rendimiento ---
AsNoTracking: 42 ms | Órdenes: 3
AsNoTrackingWithIdentityResolution: 48 ms | Órdenes: 3
```

La primera sección muestra que con `AsNoTracking` se cargan tres órdenes y cuatro instancias de planchas. La segunda sección muestra que con `AsNoTrackingWithIdentityResolution` se cargan tres órdenes y cuatro instancias de planchas. La tercera sección muestra que `AsNoTracking` es ligeramente más rápido.

Observaciones: en este caso, el número de instancias es el mismo porque cada plancha pertenece a una sola orden. Para ver la diferencia, se necesitaría un escenario donde la misma entidad aparezca en varias filas del resultado, como una consulta con dos colecciones incluidas. La diferencia de tiempo es pequeña porque la tabla tiene pocas filas.

Paso 10: Diagnosticar un error común

Modificar el método `ContarPlanchasInstanciadas` para usar `HashSet` sin `ReferenceEqualityComparer`:

```
public int ContarPlanchasInstanciadas(List<OrdenFabricacion> ordenes)
{
    var instancias = new HashSet<PlanchaAcero>();
    foreach (var orden in ordenes)
    {
        foreach (var plancha in orden.Planchas)
        {
            instancias.Add(plancha);
        }
    }
    return instancias.Count;
}
```

Resultado esperado: el método usa `Equals` y `GetHashCode` de `PlanchaAcero`, que no están sobrescritos. Como `PlanchaAcero` no sobrescribe `Equals`, el `HashSet` compara por referencia y cuenta las instancias correctamente. Sin embargo, si `PlanchaAcero` sobrescribiera `Equals` por valor, el `HashSet` contaría las planchas por valor y no por referencia.

Solución: usar `ReferenceEqualityComparer.Instance` para garantizar que la comparación sea por referencia.

```
var instancias = new HashSet<PlanchaAcero>(ReferenceEqualityComparer.Instance);
Resultado esperado con la solución: el HashSet compara por referencia y cuenta las instancias correctamente.
```

Errores comunes del ejercicio

Error	Causa	Solución
Instancias duplicadas	Se usó AsNoTracking con relaciones	Usar AsNoTrackingWithIdentityResolution
Número de instancias incorrecto	Se usó HashSet sin ReferenceEqualityComparer	Usar ReferenceEqualityComparer.Instance
Rendimiento degradado	Se usó AsNoTrackingWithIdentityResolution sin necesidad	Usar AsNoTracking si no hay duplicados
Caché de identidad compartida	Se usó el mismo DbContext para varias consultas	Usar un DbContext distinto por consulta
Proyección con resolución de identidad	Se aplicó a una proyección	No tiene efecto en proyecciones

Reto resuelto: Comparar instancias con dos colecciones incluidas

Reto: Crear un método en el repositorio que cargue las órdenes con dos colecciones incluidas (planchas y entidades intermedias) usando AsNoTracking y AsNoTrackingWithIdentityResolution. Comparar el número de instancias de aleaciones en cada caso.

Solución paso a paso:

Paso 1: Añadir los métodos a la interfaz IOrdenRepositorio:

```
List<OrdenFabricacion> ObtenerConDosColeccionesAsNoTracking();
List<OrdenFabricacion> ObtenerConDosColeccionesConResolucionIdentidad();
```

Línea 1: List ObtenerConDosColeccionesAsNoTracking(); → declara el método con AsNoTracking.

Línea 2: List ObtenerConDosColeccionesConResolucionIdentidad(); → declara el método con resolución de identidad.

Paso 2: Implementar los métodos en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerConDosColeccionesAsNoTracking()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerConDosColeccionesConResolucionIdentidad()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerConDosColeccionesAsNoTracking() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 6: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 7: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 8: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 9: `.ToList();` → materializa la consulta.

Línea 12: `public List ObtenerConDosColeccionesConResolucionIdentidad()` → declara el método.

Línea 14: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 15: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 16: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 17: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 18: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 19: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 20: `.ToList();` → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarDosColecciones()
{
    Console.WriteLine("\n--- Dos colecciones: AsNoTracking ---");
    var ordenes1 = _unidad.Ordenes.ObtenerConDosColeccionesAsNoTracking();
    var aleaciones1 = new HashSet<Aleacion>(ReferenceEqualityComparer.Instance);
    foreach (var orden in ordenes1)
    {
        foreach (var oa in orden.OrdenesAleaciones)
        {
            aleaciones1.Add(oa.Aleacion);
        }
    }
    Console.WriteLine($"Instancias de aleaciones: {aleaciones1.Count}");

    Console.WriteLine("\n--- Dos colecciones: AsNoTrackingWithIdentityResolution ---");
    var ordenes2 = _unidad.Ordenes.ObtenerConDosColeccionesConResolucionIdentidad();
    var aleaciones2 = new HashSet<Aleacion>(ReferenceEqualityComparer.Instance);
    foreach (var orden in ordenes2)
    {
        foreach (var oa in orden.OrdenesAleaciones)
        {
            aleaciones2.Add(oa.Aleacion);
        }
    }
    Console.WriteLine($"Instancias de aleaciones: {aleaciones2.Count}");
}
```

Línea 1: `private void DemostrarDosColecciones()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Dos colecciones: AsNoTracking ---");` → muestra la cabecera.

Línea 4: `var ordenes1 = _unidad.Ordenes.ObtenerConDosColeccionesAsNoTracking();` → carga las órdenes con `AsNoTracking`.

Línea 5: `var aleaciones1 = new HashSet(ReferenceEqualityComparer.Instance);` → crea un conjunto de instancias por referencia.

Línea 6: `foreach (var orden in ordenes1)` → itera sobre las órdenes.

Línea 8: `foreach (var oa in orden.OrdenesAleaciones)` → itera sobre las entidades intermedias.

Línea 10: `aleaciones1.Add(oa.Aleacion);` → añade la aleación al conjunto.

Línea 13: `Console.WriteLine($"Instancias de aleaciones: {aleaciones1.Count}");` → muestra el número de instancias.

Línea 15: `Console.WriteLine("\n--- Dos colecciones: AsNoTrackingWithIdentityResolution ---");` → muestra la cabecera.

Línea 16: `var ordenes2 = _unidad Ordenes.ObtenerConDosColeccionesConResolucionIdentidad();` → carga las órdenes con resolución de identidad.

Línea 17: `var aleaciones2 = new HashSet<Aleacion>(ReferenceEqualityComparer.Instance);` → crea un conjunto de instancias por referencia.

Línea 18: `foreach (var orden in ordenes2)` → itera sobre las órdenes.

Línea 20: `foreach (var oa in orden.OrdenesAleaciones)` → itera sobre las entidades intermedias.

Línea 22: `aleaciones2.Add(oa.Aleacion);` → añade la aleación al conjunto.

Línea 25: `Console.WriteLine($"Instancias de aleaciones: {aleaciones2.Count}");` → muestra el número de instancias.

| Paso 4: Llamar al método desde Ejecutar:

```
DemostrarDosColecciones();
```

| Paso 5: Ejecutar dotnet run y comparar los resultados.

Resultado esperado: con `AsNoTracking`, el número de instancias de aleaciones es mayor porque la misma aleación se instancia varias veces. Con `AsNoTrackingWithIdentityResolution`, el número de instancias es menor porque las aleaciones se resuelven a la misma instancia.

| Analogía final

La resolución de identidad en una acería es como el fichero de planchas que el supervisor mantiene durante una inspección. Cuando se inspeccionan varias órdenes, el supervisor anota cada plancha que ve. Sin resolución de identidad, el supervisor anota la misma plancha cada vez que la ve en una orden distinta: al final, tiene varias fichas de la misma plancha. Con resolución de identidad, el supervisor comprueba si la plancha ya está anotada antes de anotarla de nuevo: al final, tiene una sola ficha por plancha. La caché de identidad temporal es como el fichero que el supervisor usa durante la inspección y destruye al terminar. La caché de identidad del Change Tracker es como el fichero permanente que el supervisor mantiene durante todo el turno. `AsNoTracking` es como inspeccionar sin fichero.

`AsNoTrackingWithIdentityResolution` es como inspeccionar con un fichero temporal. `AsTracking` es como inspeccionar con el fichero permanente. Así funciona la resolución de identidad en EF Core: se elige la variante según el escenario y el coste que se está dispuesto a asumir.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de resolución de identidad a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `ResolucionIdentidadUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con órdenes y planchas.

Ejecutado las demostraciones de `AsNoTracking` y `AsNoTrackingWithIdentityResolution`.

Comparado el rendimiento de ambas variantes.

Diagnosticado el error de usar HashSet sin ReferenceEqualityComparer.

Creado los métodos ObtenerConDosColeccionesAsNoTracking y
ObtenerConDosColeccionesConResolucionIdentidad.
